

Day 4 - CSS Layout Fundamentals: Display, Position, Margin Collapsing

Day 4 — CSS Layout Fundamentals: Display, Position, and Margin Collapsing

Objectives

- Understand the `display` property and the difference between block, inline, and inline-block elements
- Understand `position` and the different "positioning contexts" it creates
- Understand a classic beginner-confusing behavior: margin collapsing
- Build enough of a foundation here that Flexbox (Day 5) and Grid (Day 6) will make immediate sense as *solutions* to problems this day's tools can't solve well

`display` — how an element behaves in the normal flow of the page

Every HTML element has a default `display` value that determines two things: whether it starts on its own new line, and whether you're allowed to set a `width/height` on it directly. The three values you'll use constantly:

block — takes up the *entire width available* to it, and always starts on a brand new line, pushing anything after it down below. You CAN set `width/height` on a block element directly. Elements like `<div>`, `<p>`, `<h1>`-`<h6>`, `<ul>`, `<li>` are **block** by default.

```
div { display: block; } /* this is already the default for div -- shown explicitly for clarity */
```

inline — takes up *only as much width as its content needs*, sits inline with surrounding text (doesn't force a new line), and — this is the part that trips people up — **setting `width` or `height` on an inline element has no effect at all**; the browser simply ignores it. Elements like `<a>`, `<span>`, `<strong>`, `<em>` are **inline** by default.

```
<p>This is <span style="width: 500px;">some text</span> in a paragraph.</p>
<!-- the width: 500px above is silently IGNORED, because span is inline by default -->
```

inline-block — the useful hybrid: behaves like **inline** in that it doesn't force a new line and sits alongside other content, but, unlike plain **inline**, it DOES respect `width/height` that you set on it. This is genuinely useful whenever you want several items sitting next to each other, each with a controlled size — though, as you'll discover on Day 5, Flexbox is usually a cleaner, more powerful tool for that specific goal, and **inline-block** is used less often in modern code as a result. It's still worth understanding, since you'll encounter it in existing codebases.

A fourth value worth knowing: **none** — completely removes the element from the page, as if it didn't exist at all (it takes up no space whatsoever, unlike merely making something invisible, which you'll meet with the separate `visibility` property later in this course).

```
.hidden { display: none; }
```

`position` — taking an element out of (or adjusting it within) the normal flow

By default, every element has `position: static` — meaning it simply sits in the normal, top-to-bottom, left-to-right document flow, exactly where its HTML places it, unaffected by any `top/left/right/bottom` properties (which only work once you change `position` away from `static`). Four other values change this behavior fundamentally:

relative — the element still takes up its normal spot in the flow (nothing else moves to compensate), but you can now nudge it visually using `top/left/right/bottom`, offset from where it would otherwise have sat:

```
.nugged {
  position: relative;
  top: 10px;    /* moves it 10px DOWN from its normal position */
  left: 20px;   /* moves it 20px RIGHT from its normal position */
}
```

Note this only shifts the *visual* rendering — the space the element originally occupied in the flow is still reserved, unaffected, which can create visual overlap with surrounding elements. `position: relative` has one more critical job, described next.

absolute — completely removes the element from the normal flow (surrounding elements behave as if it isn't there at all, collapsing into the space it would have taken), and positions it using `top/left/right/bottom` measured relative to its **nearest positioned ancestor** — meaning the closest parent element (however many levels up) that has a `position` value other than `static`. If no ancestor at all has a non-static position, it positions relative to the entire page instead.

```
.container {
  position: relative;    /* this becomes the "positioned ancestor" for the child below */
}
.badge {
  position: absolute;
  top: 5px;
  right: 5px;    /* pins this element to the top-right CORNER of .container, not the whole page */
}
```

This is precisely *why* you'll often see a seemingly-pointless `position: relative;` (with no `top/left` values at all) on a parent element — its entire purpose is purely to establish that parent as the reference point ("positioning context") for an absolutely-positioned child inside it. Without it, that child would instead position itself relative to the entire page — rarely what you actually want.

fixed — also removed from the normal flow, but positioned relative to the browser window itself, and stays anchored to that exact spot on screen even as the user scrolls the page. Common use: a navigation bar that stays visible at the top of the screen no matter how far down the page you've scrolled.

sticky — a hybrid: behaves like **relative** (in the normal flow) until the page scrolls to the point where it would go offscreen, at which point it "sticks" and behaves like **fixed** for as long as its containing parent is still on screen. A very common real use: a section heading that stays pinned to the top while you scroll through that section's content, then scrolls away normally once you reach the next section.

Margin collapsing — a classic "why is there extra space here" gotcha

Here's a genuinely surprising behavior that confuses nearly every beginner at least once: when two **block** elements are stacked vertically, and the first one has a **margin-bottom**, and the second has a **margin-top**, **the two margins do not simply add together — the browser collapses them into a single margin, equal to whichever of the two values was larger** (not their sum):

```
.first { margin-bottom: 30px; }
.second { margin-top: 20px; }

<div class="first">First box</div>
<div class="second">Second box</div>
```

You might expect the visible gap between these two boxes to be $30\text{px} + 20\text{px} = 50\text{px}$. It's actually just **30px** — the larger of the two, not the sum. This is called **margin collapsing**, and it only happens for *vertical* margins between block-level elements in the normal document flow (it does NOT happen for padding, for horizontal margins, or for elements inside a Flexbox/Grid container, which you'll meet the next two days — this is one of several small, real reasons Flexbox and Grid feel more predictable to work with once you get there). Knowing this rule exists — even before you've fully internalized every edge case of exactly when it applies — is what turns "why is this spacing wrong, this is so confusing" into "oh, that's margin collapsing" the moment you see it happen in your own work.

Exercises

Open `starter.html` and `starter.css`, follow the `<!-- TODO -->` `/* TODO */` markers, and check your work by reloading in the browser and comparing against `CHECKLIST.md`.